Foundations of BERT language model

[BERT language model]

1.0 Content Block

Embedding This section describes the embedding used by BERTBASE. The other one, BERTLARGE, is similar, just larger. The tokenizer of BERT is WordPiece, which is a sub-word strategy like byte-pair encoding. Its vocabulary size is 30,000, and any token not appearing in its vocabulary is replaced by [UNK] ("unknown").

2.0 Content Block

GLUE (General Language Understanding Evaluation) task set (consisting of 9 tasks); SQuAD (Stanford Question Answering Dataset) v1.1 and v2.0; SWAG (Situations With Adversarial Generations). In the original paper, all parameters of BERT are fine-tuned, and recommended that, for downstream applications that are text classifications, the output token at the [CLS] input token is fed into a linear-softmax layer to produce the label outputs. The original code base defined the final linear layer as a "pooler layer", in analogy with global pooling in computer vision, even though it simply discards all output tokens except the one corresponding to [CLS] .

3.0 Content Block

BERT is meant as a general pretrained model for various applications in natural language processing. That is, after pre-training, BERT can be fine-tuned with fewer resources on smaller datasets to optimize its performance on specific tasks such as natural language inference and text classification, and sequence-to-sequence-based language generation tasks such as question answering and conversational response generation. The original BERT paper published results demonstrating that a small amount of finetuning (for BERTLARGE, 1 hour on 1 Cloud TPU) allowed it to achieved state-of-the-art performance on a number of natural language understanding tasks:

4.0 Content Block

replaced with a [MASK] token with probability 80%, replaced with a random word token with probability 10%, not replaced with probability 10%. The reason not all selected tokens are masked is to avoid the dataset shift problem. The dataset shift problem arises when the distribution of inputs seen during training differs significantly from the distribution encountered during inference. A trained BERT model might be applied to word representation (like Word2Vec), where it would be run over sentences not containing any [MASK] tokens. It is later found that more diverse training objectives are generally better. As an illustrative example, consider the sentence "my dog is cute". It would first be divided into tokens like "my1 dog2 is3 cute4". Then a random token in the sentence would be picked. Let it be the 4th one "cute4". Next, there would be three possibilities:

5.0 Content Block

Given two sentences, the model predicts if they appear sequentially in the training corpus, outputting either [IsNext] or [NotNext]. During training, the algorithm sometimes samples two sentences from a single continuous span in the training corpus, while at other times, it samples two sentences from two discontinuous spans. The first sentence starts with a special token, [CLS] (for "classify"). The two sentences are separated by another special token, [SEP] (for "separate"). After processing the two sentences, the final vector for the [CLS] token is passed to a linear layer for binary classification into [IsNext] and [NotNext]. For example:

6.0 Content Block

Masked language modeling (MLM): In this task, BERT ingests a sequence of words, where one word may be randomly changed ("masked"), and BERT tries to predict the original words that had been changed. For example, in the sentence "The cat sat on the [MASK]," BERT would need to predict "mat." This helps BERT learn bidirectional context, meaning it understands the relationships between words not just from left to right or right to left but from both directions at the same time. Next sentence prediction (NSP): In this task, BERT is trained to predict whether one sentence logically follows another. For example, given two sentences, "The cat sat on the mat" and "It was a sunny day", BERT has to decide if the second sentence is a valid continuation of the first one. This helps BERT understand relationships between sentences, which is important for tasks like question answering or document classification.

7.0 Content Block

The three attention matrices are added together element-wise, then passed through a softmax layer and multiplied by a projection matrix. Absolute position encoding is included in the final self-attention layer as additional input.

8.0 Content Block

Tokenizer: This module converts a piece of English text into a sequence of integers ("tokens"). Embedding: This module converts the sequence of tokens into an array of real-valued vectors representing the tokens. It represents the conversion of discrete token types into a lower-dimensional Euclidean space. Encoder: a stack of Transformer blocks with self-attention, but without causal masking. Task head: This module converts the final representation vectors into one-shot encoded tokens again by producing a predicted probability distribution over the token types. It can be viewed as a simple decoder, decoding the latent representation into token types, or as an "un-embedding layer". The task head is necessary for pre-training, but it is often unnecessary for so-called "downstream tasks," such as question answering or sentiment classification. Instead, one removes the task head and replaces it with a newly initialized module suited for the task, and finetune the new module. The latent vector representation of the model is directly fed into this new module, allowing for sample-efficient transfer learning.

9.0 Content Block

Given "[CLS] my dog is cute [SEP] he likes playing [SEP]", the model should predict [IsNext]. Given "[CLS] my dog is cute [SEP] how do magnets work [SEP]", the model should predict [NotNext].

10.0 Content Block

Architectural family The encoder stack of BERT has 2 free parameters: L {\displaystyle L} , the number of layers, and H {\displaystyle H} , the hidden size. There are always H / 64 {\displaystyle H/64} self-attention heads, and the feed-forward/filter size is always 4 H {\displaystyle 4H} . By varying these two numbers, one obtains an entire family of BERT models. For BERT:

11.0 Content Block

Interpretation Language models like ELMo, GPT-2, and BERT, spawned the study of "BERTology", which attempts to interpret what is learned by these models. Their performance on these natural language understanding tasks are not yet well understood. Several research publications in 2018 and 2019 focused on investigating the relationship behind BERT's output as a result of carefully chosen input sequences, analysis of internal vector representations through probing classifiers, and the relationships represented by attention weights. The high performance of the BERT model could also be attributed to the fact that it is bidirectionally trained. This means that BERT, based on the Transformer model architecture, applies its self-attention mechanism to learn information from a text from the left and right side during training, and consequently gains a deep understanding of the context. For example, the word fine can have two different meanings depending on the context (I feel fine today, She has fine blond hair). BERT considers

the words surrounding the target word fine from the left and right side. However it comes at a cost: due to encoder-only architecture lacking a decoder, BERT can't be prompted and can't generate text, while bidirectional models in general do not work effectively without the right side, thus being difficult to prompt. As an illustrative example, if one wishes to use BERT to continue a sentence fragment "Today, I went to", then naively one would mask out all the tokens as "Today, I went to [MASK] [MASK] [MASK] ... [MASK] ." where the number of [MASK] is the length of the sentence one wishes to extend to. However, this constitutes a dataset shift, as during training, BERT has never seen sentences with that many tokens masked out. Consequently, its performance degrades. More sophisticated techniques allow text generation, but at a high computational cost.